06.843.0611 Language AI Engineering / Programmieren für Linguisten (Übung) – Modul 1: Einführung

Sommersemester 2026

Daniel Zielinski
zielinski@uni-mainz.de

Lernziele (Modul 1)

- Daten, Methoden und Prozesse des Sprachdatenmanagements kennenlernen
- Grundlagen zu Python lernen (Was ist Python? Wozu wird es verwendet? …)
- Funktionstüchtigkeit der Python-Umgebungen prüfen
- Erste Schritte in der Programmierung mit Python machen:
  - Variablen
  - Datentypen
  - Listen
  - Dictionaries

© 2026 Loctimize GmbH – Alle Rechte vorbehalten

Sprachdatenmanagement

Daten

- Plaintext (*txt)
- Formatierter Text (*doc)
- Austauschformate (*xml, *html, *xliff)
- Translation Memories (*tmx)
- Terminologiedaten (*tbx, *csv, *xlsx)
- Audiodaten (*wav, *mp3, *flac, …)
- Videodaten (*avi, *mp4, *mkv, *webm, …)

© 2026 Loctimize GmbH – Alle Rechte vorbehalten

Methoden und Prozesse

- Segmentierung
- Tokenisierung & Lemmatisierung
- POS-Tagging (verschiedene Tagging-Standards)
- Parsing
- Information extraction & Text classification
- Text analysis & Speech analysis & Sentiment analysis
- Natural language generation
- Speech recognition & Language recognition
- ...

© 2026 Loctimize GmbH – Alle Rechte vorbehalten

Einführung in das Arbeiten mit Python

Was ist Python?

- High-Level-Programmiersprache
- Einsatzbereiche
  - Web-Frameworks
  - Natural Language Processing
  - Künstliche Intelligenz
  - …

„Andere High-Level-Programmiersprachen, von denen Sie vielleicht schon gehört haben, sind Java, C/C++ und PHP.“

© 2026 Loctimize GmbH – Alle Rechte vorbehalten

Programmiersprachen vs. Natürliche Sprachen

- Programmiersprachen sind immer eindeutig und wortwörtlich.
- Die Struktur ist sehr wichtig.
- Es kommt auf die Details an: Kleinigkeiten wie Rechtschreibfehler und schlechte Zeichensetzung können in einer Programmiersprache einen großen Unterschied machen!
- Code kann nicht einfach wie Text gelesen werden.

„Wir werden lernen, Programme im Kopf zu analysieren, die Codeteile zu identifizieren und die Struktur zu interpretieren.“

© 2026 Loctimize GmbH – Alle Rechte vorbehalten

Warum Python?

- Funktioniert auf verschiedenen Plattformen (Windows, Mac, Linux, Raspberry Pi, usw.).
- Wurde im Hinblick auf Lesbarkeit entwickelt und hat eine einfache Syntax, die der englischen Sprache ähnelt.
- Hat eine Syntax, die es Entwicklern ermöglicht, Programme mit weniger Zeilen zu schreiben als einige andere Programmiersprachen.

© 2026 Loctimize GmbH – Alle Rechte vorbehalten

Python

```python
print("Hello World")
```

Java

```java
public class Main {
    public static void main(String[] args) {
        System.out.println("Hello World");
    }
}
```

(c) 2026 Loctimize GmbH. Alle Rechte vorbehalten

Oh mein Gott!

Python vs. andere Programmiersprachen

- Python wird interpretiert, nicht kompiliert.
- Python verwendet Zeilenumbrüche, um einen Befehl abzuschließen.
  - Anderen Programmiersprachen nutzen oft Semikolons oder Klammern dafür.
- Python verwendet Einrückungen, um den Geltungsbereich von Code zu definieren.
  - In andere Programmiersprachen wird das durch {…} getan.
  - Einrückungen sind nur ein optionales Mittel, um die Lesbarkeit des Codes zu verbessern.

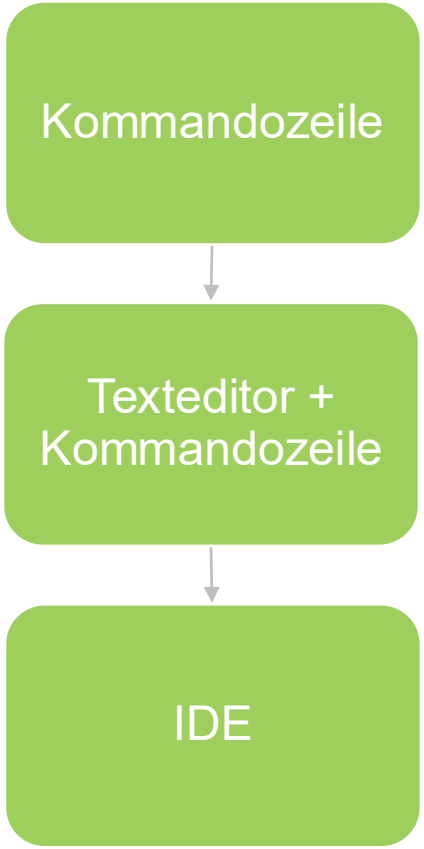
© 2026 Loctimize GmbH – Alle Rechte vorbehalten

Arbeitsumgebungen

Python

- Kann in verschiedenen Arbeitsumgebungen geschrieben werden
- Direkte Eingabebefehle oder Skripte

➢ Um eine kurze Menge an Code in Python zu testen, ist es manchmal am schnellsten und einfachsten, den Code nicht in eine Datei zu schreiben.



© 2026 Loctimize GmbH – Alle Rechte vorbehalten

Python Versionen

- Neueste Version: Python 3.14.4 (Stand 14.4.26)
- Auch heute noch im Einsatz: Python 2 (obwohl es außer Sicherheitsupdates keine weiteren Updates gibt)
- Unterschiede:
  - Keine Unterstützung versionsspezifischer Libraries
  - Code wird teilweise anders geschrieben und verarbeitet
  - Unterschiedliche Textverarbeitung (ASCII vs. Unicode)

➢ Wir benutzen Python 3!

© 2026 Loctimize GmbH – Alle Rechte vorbehalten

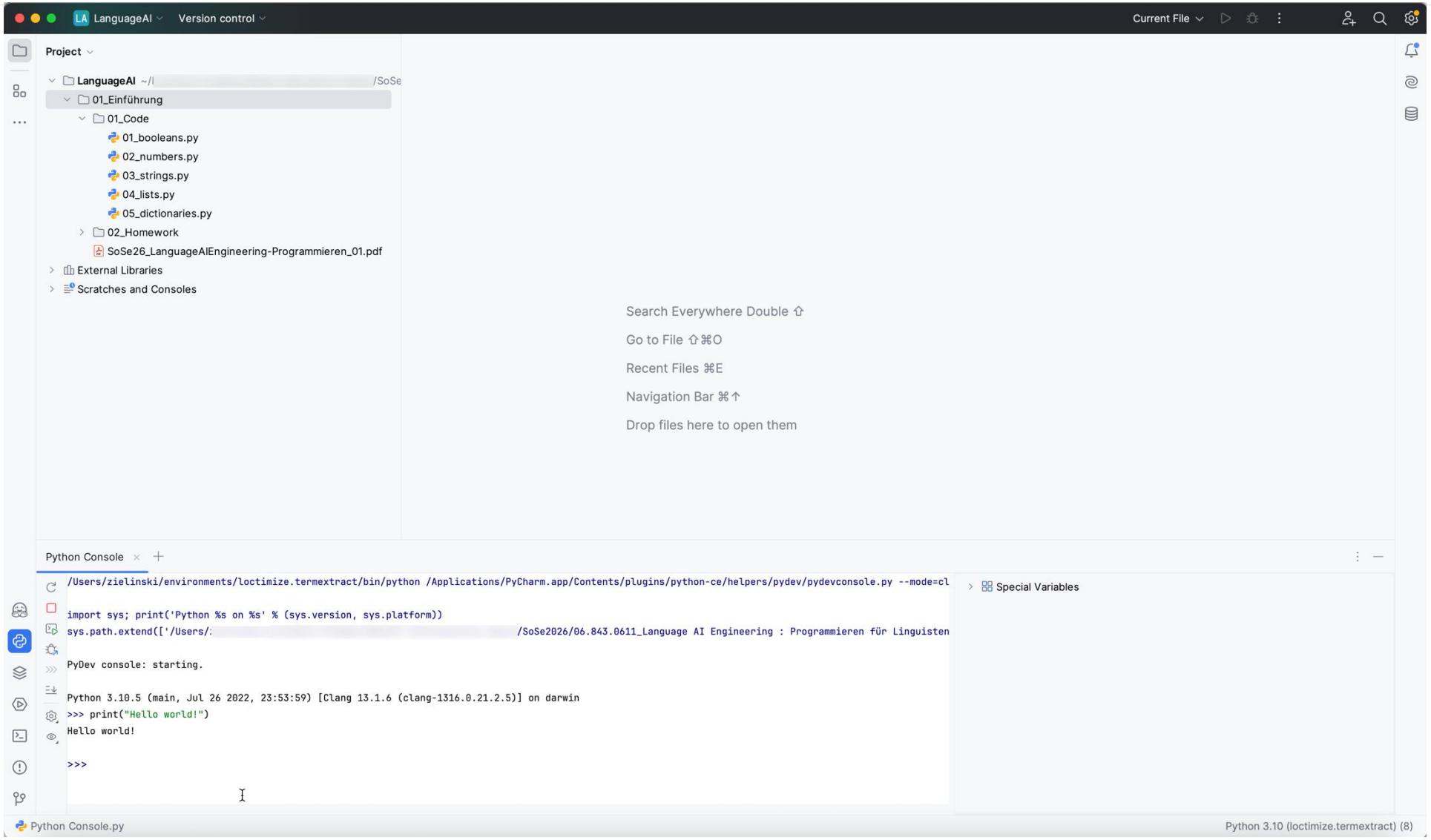
Erste Schritte

Aktivität: Arbeitsumgebung prüfen

1. *Öffnen Sie PyCharm.*
2. *Prüfen Sie, ob* `Python 3.x` *als Interpreter eingestellt ist.*
3. *Erstellen Sie einen Ordner namens* `LanguageAI`*, in dem Sie die Dateien des Moduls entpacken.*
4. *Öffnen Sie in PyCharm den Ordner als neues Projekt.*

© 2026 Loctimize GmbH – Alle Rechte vorbehalten

Language AI Engineering / Programmieren für Linguisten | Arbeitsumgebungen | PyCharm



© 2026 Loctimize GmbH – Alle Rechte vorbehalten

Variablen

- Container, in denen ein Wert gespeichert wird
- Geben den gespeicherten Wert zurück, wenn sie aufgerufen werden
- Werte werden mit einem Gleichheitszeichen zugewiesen:

Variablenname = Wert

- Müssen definiert werden, bevor sie aufgerufen werden
- Können überschrieben werden

© 2026 Loctimize GmbH – Alle Rechte vorbehalten

Variablennamen sollten kurz und aussagekräftig sein.

Sie sollten mit einem Buchstaben oder einem Unterstrich beginnen.

Sie dürfen nicht mit einer Zahl beginnen!

Sie können nur alphanumerische Zeichen und Unterstriche enthalten (A-z, 0-9 und _)

Sie dürfen nicht nach reservierten Begriffen benannt werden!

© 2026 Loctimize GmbH – Alle Rechte vorbehalten

Variablen

- WICHTIG: Python unterscheidet zwischen Groß- und Kleinschreibung!
  - age, Age und AGE sind drei verschiedene Variablen!
- Best Practice: Englisch und im Snake Case

sample_variable

© 2026 Loctimize GmbH – Alle Rechte vorbehalten

Boolesche Werte und Operatoren

Boolesche Werte

- Datentyp: *Boolean*
- Überprüfen die Richtigkeit von Aussagen
- Rückgabewert: *True* oder *False*.
- Verwendung: Grundlage von Konditionalsätzen und while-Schleifen
  - um zu entscheiden, wie ein Wert weiterverarbeitet werden soll
  - wann ein Code gestoppt werden soll

Gleich (==) Ungleich (!=)	Größer (>) Kleiner (<)	Größer gleich (>=) Kleiner gleich (<=)

© 2026 Loctimize GmbH – Alle Rechte vorbehalten

on ens257 to 192.168.0.1
from 192.168.0.1
from 192.168.0.1
bound to 192.168.0.216 — renewal in
bound to 192.168.0.216 — renewal in
[718]: <info> (ens257): DHCPv4 state
[718]: <info> address 192.168.0.216
[718]: <info> plen 24 (255.255.255.0)
[718]: <info> gateway 192.168.0.1
[718]: <info>

Datentypen

Primitive Datentypen

- Wahrheitswerte: ***bool*** *(True, False)*
- Zahlen: ***int*** (1, 2, …), ***float*** (1.5, …)
- Text: ***str*** (“Das ist ein String”, “True”, “1”, “2.5”…)

- Der Datentyp kann mit folgender Funktion kontrolliert werden:

type(Variable**)**

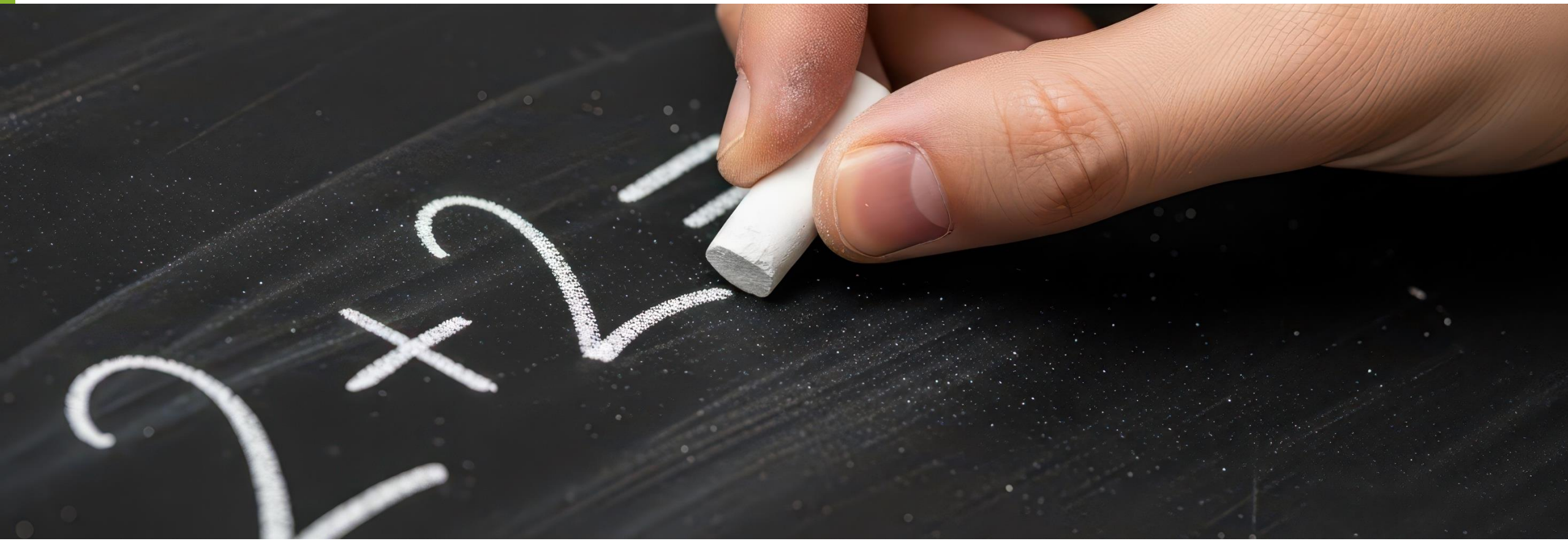
© 2026 Loctimize GmbH – Alle Rechte vorbehalten

Datentyp konvertieren

- Python bestimmt den Datentyp automatisch, d.h. bei der Erstellung von Variablen wird i.d.R. kein Datentyp angegeben
- Manchmal muss der Datentyp zu einem späteren Zeitpunkt mit expliziten Deklarationen geändert werden, dazu verwendet man folgende Funktionen:

str(Variable) | **int(Variable)** | **float(Variable)**

© 2026 Loctimize GmbH – Alle Rechte vorbehalten



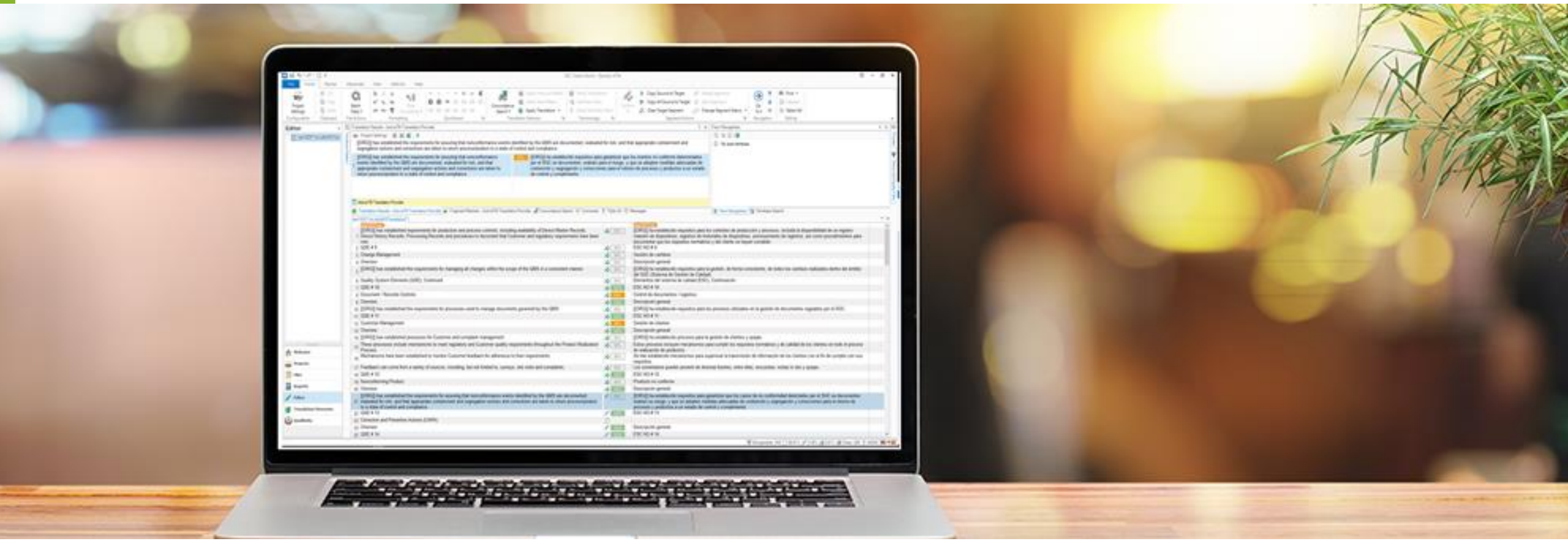
Arithmetik

Arithmetik

- Mit *Integers* und *Floats* können einfache arithmetische Operation durchgeführt werden
- Folgende arithmetische Operatoren stehen zur Verfügung:

Addition (+)
Subtraktion (-)
Multiplikation (*)
Division (/) – ACHTUNG, das Ergebnis einer Division ist immer ein Float-Wert!
Modulo (%) – Gibt den verbleibenden Rest einer Division zurück. Zum Beispiel 5 % 2 = 1

© 2026 Loctimize GmbH – Alle Rechte vorbehalten



Textverarbeitung

Strings

- Text (i.d.R. Wörter oder Sätze)
- Auch Zahlen können als Strings angegeben werden!
- Stehen in einfachen oder doppelten Anführungszeichen
- Mehrzeilige Strings haben jeweils drei einfache oder doppelte Anführungszeichen vor und nach dem String

.capitalize()

.upper()

.lower()

© 2026 Loctimize GmbH – Alle Rechte vorbehalten

Listen

Von **Monster-Strings** zu Listen

- .split() teilt den String an beliebigen Stellen und speichert die einzelnen Teile in einer Liste:

<variable>**.split()**

<variable>**.split(".")**

<variable>**.split(",")**

<variable>**.split(" und ")**

…

© 2026 Loctimize GmbH – Alle Rechte vorbehalten

Listen

- Komplexer Datentyp
- Sind umschlossen von eckigen Klammern
- Können mehrere Werte in einer Variablen speichern:

```
empty_list = []
coffee_list = ["Americano", "Espresso", "Cappuccino"]
random_list = ["coffee", "coffee", 5, ["cookies", "cake"], True, 2.3]
```

© 2026 Loctimize GmbH – Alle Rechte vorbehalten

Indexing

Element

1 2 3 4 5 6

```
sample_list = [“Python”, “is”, “an”, “interpreted”, “programming”, “language”]
```

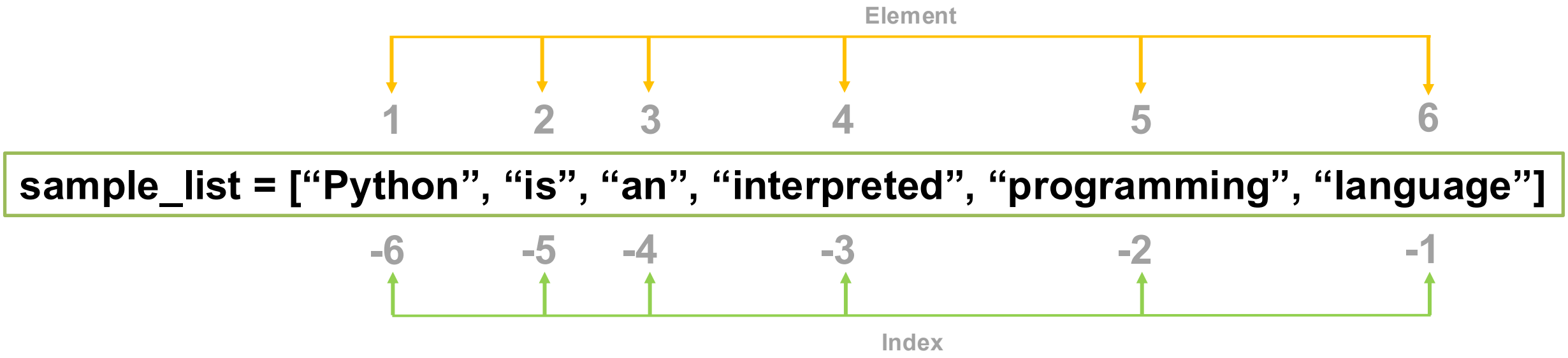
0 1 2 3 4 5

Index

```
sample_list[0]
```

© 2026 Loctimize GmbH – Alle Rechte vorbehalten

Negativ Indexing




```
sample_list[-1]
```

© 2026 Loctimize GmbH – Alle Rechte vorbehalten

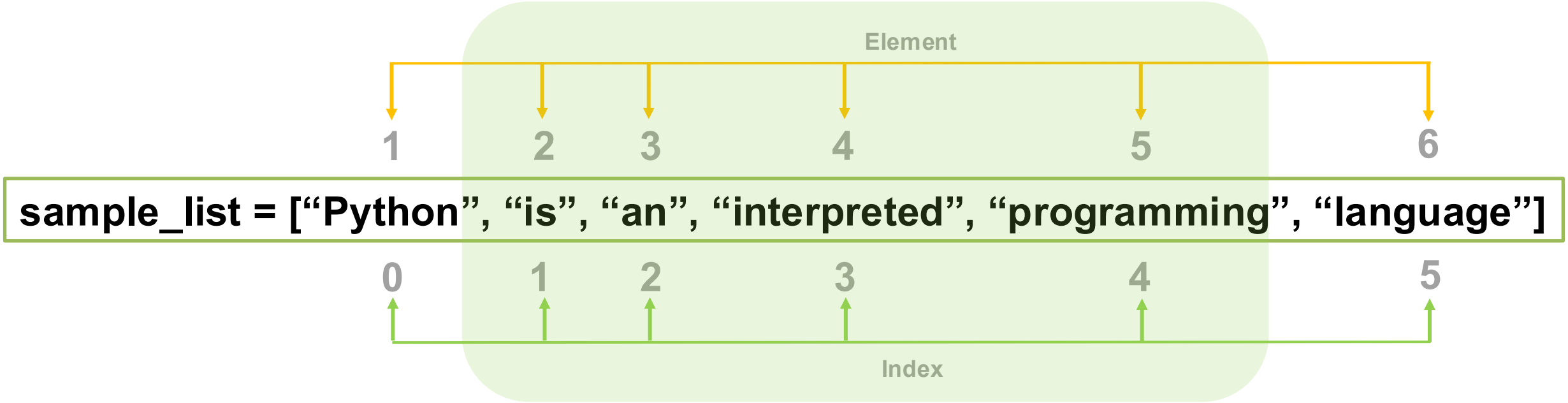
Eingebaute Methoden

- So wie Strings verfügen auch Listen über eingebaute Methoden
- z.B.:

.append()	Fügt ein neues Element ans Ende einer Liste an
.pop()	Entfernt das letzte Element einer Liste
.clear()	Entfernt alle Elemente aus einer Liste
.remove()	Entfernt ein spezifisches Element aus der Liste

© 2026 Loctimize GmbH – Alle Rechte vorbehalten

Beispiel: Element 2-5 einer Liste



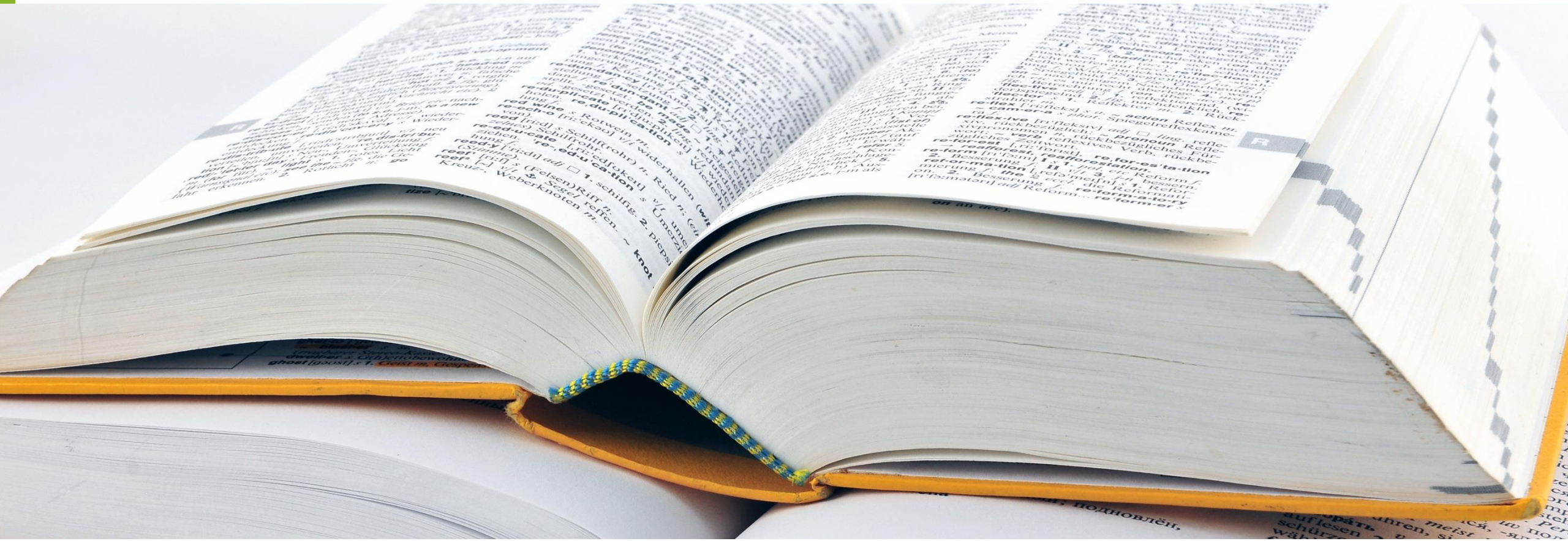
sample_list[1:5]

© 2026 Loctimize GmbH – Alle Rechte vorbehalten

Slicing Optionen

- **[Startindex:Endindex+1]** → zeigt einen begrenzten Ausschnitt
- **[:Endindex+1]** → offener Anfang: zeigt alles, was vor der zweiten Zahl steht
- **[Startindex:]** → offenes Ende: zeigt alles, ab dem angegebenen Anfangspunkt
- **[:]** → offener Anfang und offenes Ende: zeigt alles
- **[Startindex:-1]** → Das letzte Element nicht enthalten!

© 2026 Loctimize GmbH – Alle Rechte vorbehalten



Dictionaries

Dictionaries

- Komplexer Datentyp: *Dict*
- Sammlung von Daten, die keine Duplikate enthält
- Werden mit geschweiften Klammern definiert
- Daten werden in **Key:Value-Pairs** gespeichert, sind geordnet und veränderbar
- Jedem *Key* wird ein *Value* zugeordnet → Keys sind einzigartig, Values können gleich sein.

```
empty_dict = {}
dict_en-fr = {“Monday”:“lundi”, “coffee”:“café”, “good morning”:“bonjour”}
inventory = {“apples”:2, “oranges”:5, “bananas”:5, “pineapples”:1}
```

© 2026 Loctimize GmbH – Alle Rechte vorbehalten

Auf Keys und Values zugreifen

Key 1 Value 1

```
sample_dict  =  { “name” : “Jacky Daniels”,
                  “hobbies” : [“drinking”, “singing”, “telling jokes”] }
```

Key 2 Value 2

- **Keys sind wie Indexe** → **sample_dict[“name”]**
- **<dict>.keys()** gibt alle Keys in Form einer Liste zurück
- **<dict>.values()** gibt alle Keys in Form einer Liste zurück

© 2026 Loctimize GmbH – Alle Rechte vorbehalten

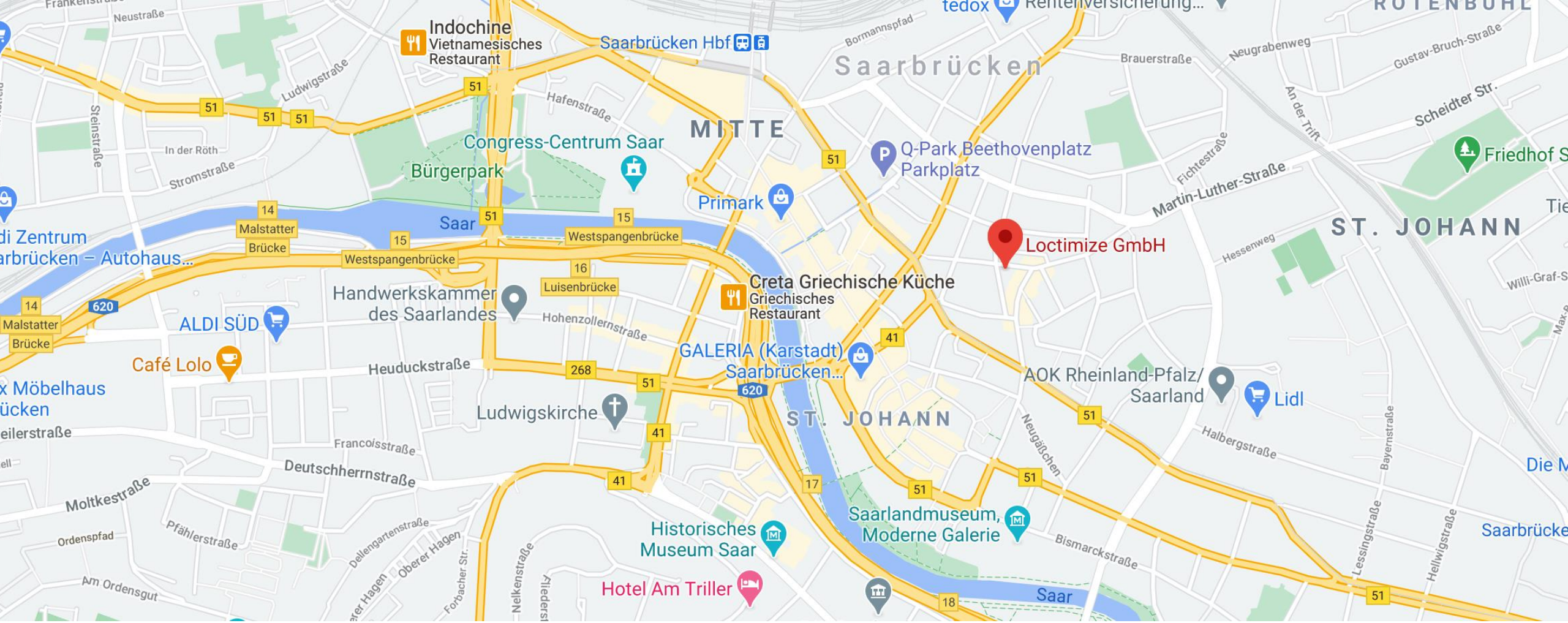
Hausaufgabe:

Experimentieren Sie mit den Inhalten dieses ersten Moduls! Testen Sie die Möglichkeiten zur Manipulation von Daten, die Python ihnen bietet.

© 2026 Loctimize GmbH – Alle Rechte vorbehalten

Vielen Dank für Ihre Aufmerksamkeit!

Bis zum nächsten Mal!



Loctimize GmbH

Försterstraße 19
66111 Saarbrücken
Germany

t. +49 681 - 91038365
f. +49 681 - 91038367
e. info@loctimize.com
w. loctimize.com